

Acelerador Parametrizável para Multiplicação de Matrizes em FPGA

Estudo de caso usando DE0-CV e comunicação UART

Fernando Schettini
fernandoschettini@outlook.com

Estudo de caso e metodologia para exploração arquitetural em FPGA

2026

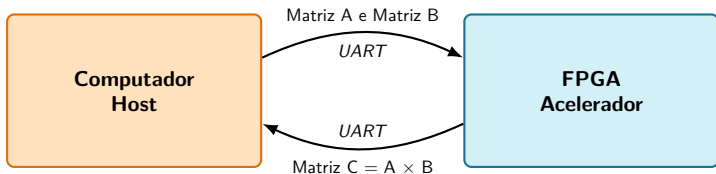
- FPGAs são alternativas para offloading em sistemas embarcados e de borda, mas o desenvolvimento de aceleradores ainda depende de fluxos, plataformas e integrações específicas^{haris2021secda, haris2023secdatflite}.
- Este trabalho explora uma arquitetura simples, parametrizável e mensurável para aceleração em FPGA de pequeno porte, composta por comunicação host-FPGA, armazenamento externo, buffers internos, unidades MAC e contadores de desempenho.

Introdução - Por que multiplicação de matrizes?

- A multiplicação de matrizes é um kernel recorrente em IA, processamento de sinais e computação científica; em deep learning, operações do tipo GEMM aparecem em camadas densas, convolucionais e recorrentes^{nvidia2023gemm}.
- O uso de matrizes 128×128 com operandos INT8 representa uma carga compatível com inferência embarcada, em que a quantização inteira reduz tráfego de memória, latência e consumo de energia^{tensorflow2022quantization}.

Introdução - Objetivo

- Desenvolver um acelerador parametrizável de multiplicação de matrizes em FPGA usando a placa DE0-CV.
- Usar o PC como host, enviando matrizes para a FPGA por UART.
- Usar a FPGA como coprocessador para executar o cálculo.



Fundamentação Teórica - Definição do Problema

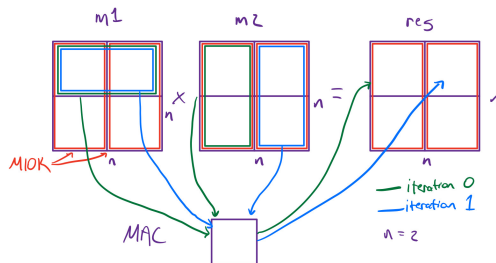


Figura 1: Representação conceitual de unidades de multiplicação e acumulação.

- Para duas matrizes A e B , a matriz resultado C é calculada por:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}$$

Fundamentação Teórica - LABs, ALMs e caminhos locais

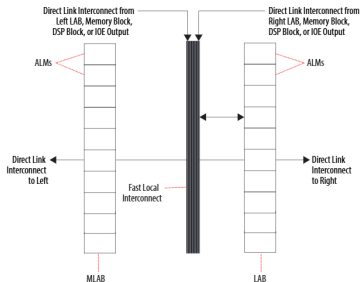


Figura 2: Estrutura de LAB no Cyclone V ^{altera2023cyclonev}.

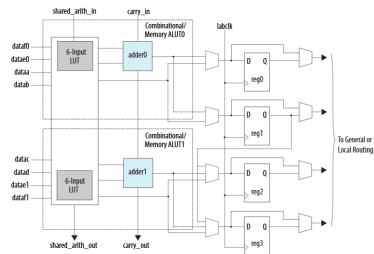


Figura 3: Estrutura de ALM no Cyclone V ^{altera2023cyclonev}.

- LABs agrupam ALMs e interconexões locais ^{altera2023cyclonev}.
- ALMs implementam LUTs, registradores, somadores e caminhos de carry ^{altera2023cyclonev}.
- *Logic element* é uma unidade lógica composta por LUT de quatro entradas, registrador programável e cadeia de carry ^{intel2017logicelement}.

Fundamentação Teórica - Interconexões no Cyclone V

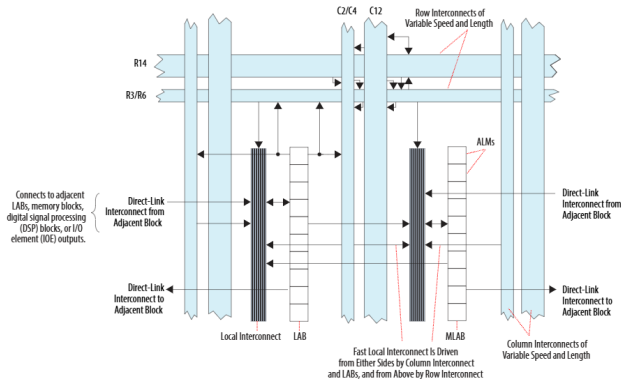


Figura 4: Interconexões locais, diretas, por linha e por coluna no Cyclone V altera2023cyclonev.

- As interconexões locais, diretas, por linha e por coluna afetam o roteamento do circuito altera2023cyclonev.

- PLLs permitem gerar clocks internos a partir do clock base da placa.
- São usados para ajustar a frequência de operação da lógica do acelerador e da interface SDRAM.

Fundamentação Teórica - DSPs

- DSPs são blocos especializados para operações aritméticas, principalmente multiplicações.
- Neste trabalho, eles são explorados por meio de unidades MAC paralelas.

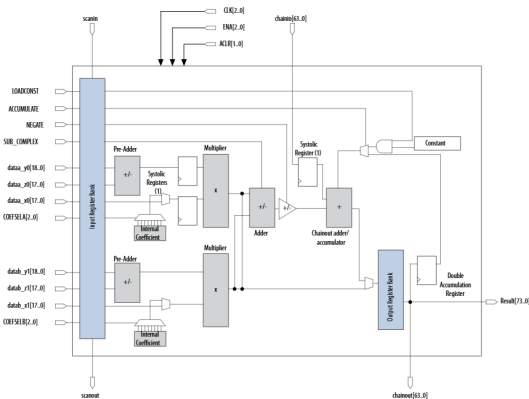


Figure 5: Diagrama interno de um bloco DSP de precisão variável no Cyclone

Tabela 1: Modos de operação dos blocos DSP de precisão variável no Cyclone
valtera2023cyclonev

Recurso	Modo	Instâncias	Pre-adder	Coeficiente	Chainout
1 DSP	9×9 independente	3	Não	Não	Não
1 DSP	18×18 independente	2	Sim	Sim	Não
1 DSP	18×19 independente	2	Sim	Sim	Não
1 DSP	18×25 independente	1	Sim	Sim	Sim
1 DSP	20×24 independente	1	Sim	Sim	Sim
1 DSP	27×27 independente	1	Sim	Sim	Sim
1 DSP	Dois 18×19 em modo adder	1	Sim	Sim	Sim
1 DSP	18×18 somado com entrada de 36 bits	1	Sim	Não	Sim
2 DSPs	Multiplicação complexa 18×19	1	Não	Não	Não

- Blocos M10K são memórias embarcadas internas da FPGA dentro do chip.
- SDRAM é memória externa à FPGA, usada como armazenamento principal de maior capacidade.
- A arquitetura usa SDRAM para matrizes completas e M10K como buffers locais de tiles.

- A placa DE0-CV tem FPGA **Cyclone V 5CEBA4F23C7N**^{terasic2015de0cv}.
- **49 mil** elementos lógicos programáveis, **18.480** Adaptive Logic Modules e **3080 Kbits** de memória embarcada^{terasic2015de0cv}.
- 66 DSPs^{terasic2015de0cv}.
- 4 PLLs fracionários^{terasic2015de0cv}.
- 64 MB de SDRAM externa, com barramento de dados de 16 bits^{terasic2015de0cv}.

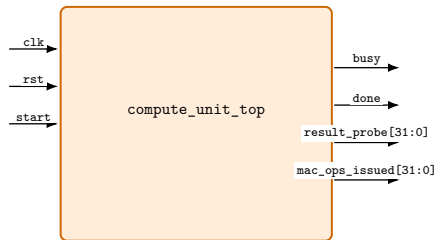
- Multiplicação densa de matrizes inteiras:

$$C = A \times B$$

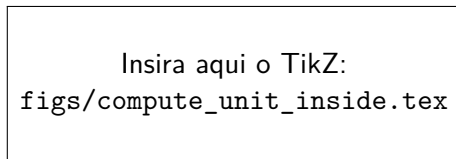
- Entradas A e B em **INT8**; acumulador e saída C em **INT32**.
- Kernel baseado em tiles e parametrizado por `TILE_SIZE`, `NUM_MACS`, `DATA_WIDTH` e `ACC_WIDTH`.
- Uso de MACs paralelas para explorar os DSPs da FPGA.

Metodologia - Fluxo local de cálculo

- Carregar tile de A .
- Carregar tile de B .
- Acumular os produtos no tile de C .
- Escrever o resultado parcial ou final.



(a) Interface externa da `compute_unit_top`.



(b) Organização interna da `compute_unit_top`.

- A primeira fase otimiza apenas a compute unit.
- Objetivo: encontrar a configuração com maior GOPS efetivo.
- Cada experimento pertence a uma tupla discreta:

$(\text{TILE_SIZE}, \text{NUM_MACS}, \text{MAC_PIPELINE_STAGES})$

- Foram avaliados:
 - TILE_SIZE em $\{2, 4, 8, 16\}$;
 - NUM_MACS em $\{1, 2, 4, 8, 16, 32, 64\}$, conforme o tamanho do tile;
 - MAC_PIPELINE_STAGES em $\{0, 1, 2\}$, nas configurações selecionadas.

Metodologia - Como o GOPS é calculado

- Para matriz quadrada $N \times N$, usa-se a aproximação:

$$ops_{approx} = 2N^3$$

- Para $N = 128$:

$$ops_{approx} = 2 \times 128^3 = 4.194.304 \text{ operações}$$

- Tempo de execução interno:

$$exec_time = \frac{exec_cycles}{f_{max}}$$

- GOPS efetivo:

$$GOPS_{eff} = \frac{ops_{approx}}{exec_time \times 10^9}$$

- Para cada configuração, o fluxo automatizado executa:
 - simulação;
 - compilação no Quartus;
 - extração de Fmax;
 - extração de ALMs, DSPs e memória;
 - geração de CSV final para comparação.

- As matrizes completas são armazenadas na SDRAM externa.
- Os tiles locais são armazenados em memórias internas M10K.
- A SDRAM atua como memória principal.
- As M10K funcionam como buffers rápidos próximos ao compute core.
- Organização em *row-major*:

$$addr = base + row \times N + col$$

- O espaço de endereçamento é dividido em regiões separadas para:
 - matriz A ;
 - matriz B ;
 - matriz C .
- Essa separação simplifica o controle de leitura, escrita e validação dos resultados.

- Segunda fase do `parameter_optimization`.
- Objetivo: avaliar o impacto da organização de memória no desempenho final.
- Cada experimento pertence a uma tupla discreta:

(`MEMORY_BURST_LEN`, `BUFFERING_MODE`, `MEMORY_BANKS_A`, `MEMORY_BANKS_B`)

- A varredura avalia 9 configurações pré-definidas.
- O modo `single` foi testado com 1 banco por matriz; o modo `double` foi testado com 2 ou 4 bancos.

- As configurações são comparadas por:
 - GOPS efetivo;
 - Fmax;
 - uso de ALMs;
 - uso de DSPs;
 - uso de memória interna;
 - pressão de roteamento.

- Parametrização da arquitetura por generics.
- Separação entre top-level, compute, memória, controle e UART.
- Uso de SDRAM externa para matrizes completas.
- Uso de M10K como buffers locais de tiles.
- Uso de FIFO na UART para reduzir perda de bytes.
- PLL para clock da SDRAM.

- Leitura de *C* por checksum para reduzir o gargalo da UART.
- Suporte a *single buffering* e *double buffering*.
- Scripts automáticos para sweeps e coleta de métricas.
- Geração de CSV, gráficos e relatórios comparativos.

Resultados - Top 3 da Compute Unit

Tabela 2: Melhores configurações do sweep `compute_unit_only`, ordenadas por $GOPS_{eff}$ aproximado.

Run	Tile	MACs	Pipe	Fmax	DSPs	ALMs	GOPS	Risco
run_013	16	64	0	65,13 MHz	64	8144	7,9629	alto
run_017	16	64	1	74,68 MHz	64	8156	7,3705	alto
run_018	16	64	2	79,23 MHz	64	8148	6,5558	alto

- O melhor desempenho puro veio de `TILE_SIZE=16` e `NUM_MACS=64`.
- Essa configuração usa praticamente todos os DSPs disponíveis e apresenta maior pressão de roteamento.

Resultados - Top 3 do acelerador completo

Tabela 3: Melhores configurações do sweep com SDRAM/top completo. A linha verde indica a configuração escolhida para teste na placa.

Run	Buffer	Burst	Bancos A	Bancos B	Fmax	ALMs	Mem. bits	GOPS
run_005	double	4	2	2	72,00 MHz	3433	68608	0,1613
run_006	double	8	2	2	72,00 MHz	3433	68608	0,1613
run_007	double	16	2	2	72,00 MHz	3433	68608	0,1613

- Configuração escolhida: N=128, TILE_SIZE=8, NUM_MACS=32, double buffering, MEMORY_BURST_LEN=4, 2 bancos para A e 2 bancos para B.
- O melhor compute puro não foi escolhido diretamente porque o teste final inclui controle, SDRAM e UART.

Resultados - Sweep previsto versus placa

Tabela 4: Comparação entre o resultado previsto pelo sweep e a execução real da run_20260609_143308.

Métrica	Sweep full/top	Medição interna na placa	Fim-a-fim com UART
GOPS	0,1613	0,0569	0,00125
Percentual do previsto	100%	35,3%	0,77%
Clock usado	72 MHz	50 MHz	50 MHz + UART
Ciclos internos	1.872.130	3.684.540	–
Tempo medido	26,00 ms	73,69 ms	16,84 s

- A queda para 35,3% do previsto vem de dois fatores: clock real menor e quase 2 vezes mais ciclos internos.
- A queda fim-a-fim é dominada pela UART, pois o tempo inclui carregamento e retorno dos dados pelo host.

Resultados - De onde vem a diferença de GOPS?

Modelo do sweep

- $f_{max} = 72$ MHz.
- $exec_cycles = 1.872.130$.
- $GOPS_{eff} = 0,1613$.

Execução na placa

- Clock real: 50 MHz.
- Ciclos reais: 3.684.540.
- $GOPS_{interno} = 0,0569$.

$$\frac{50}{72} \times \frac{1.872.130}{3.684.540} \approx 0,353$$

- Portanto, o desempenho interno medido corresponde a cerca de 35% do valor previsto no sweep.

Resultados - Comparação contextual com Raspberry Pi

Tabela 5: Comparação de ordem de grandeza. A linha do Raspberry Pi é referência externa de HPL/OpenBLAS em FP, não uma comparação direta com INT8.

Sistema / métrica	Tipo	Valor	Contra fim-a-fim UART	Observação
Compute unit FPGA, melhor sweep	INT8/INT32	7,9629 GOPS	6370x	Sem SDRAM/UART
Acelerador completo, sweep	INT8/INT32	0,1613 GOPS	129x	Modelo top completo
Acelerador na placa, interno	INT8/INT32	0,0569 GOPS	45,5x	Sem contar transferência total
Acelerador fim-a-fim UART	INT8/INT32	0,00125 GOPS	1x	Inclui comunicação UART
Raspberry Pi 5, HPL/OpenBLAS	FP / HPL	24–25 GFLOP/s	19.200x–20.000x	Referência contextual ^{rpi5hpl}

- A Raspberry Pi 5 usa CPU Broadcom BCM2712 quad-core Arm Cortex-A76 a 2,4 GHz^{raspberrypi5official}.
- A comparação mostra que o gargalo atual não é só a compute unit, mas o sistema completo com UART e validação.

Resultados - Por que o GOPS real ficou abaixo do esperado?

- O sweep estimou a melhor configuração full/top com:

$$GOPS_{sweep} = 0,1613$$

- Na placa, o clock real foi menor:

$$\frac{50}{72} = 0,694$$

- A execução real também gastou mais ciclos:

$$\frac{1.872.130}{3.684.540} = 0,508$$

- A queda observada é explicada por:

$$0,694 \times 0,508 = 0,353$$

Métrica	Valor
GOPS esperado no sweep	0,1613
GOPS interno medido	0,0569

- A UART não é adequada para alto throughput de dados.
- A leitura completa de C pela UART se torna gargalo em matrizes grandes.
- O uso de checksum reduz o tempo de validação no host.
- O gargalo final pode migrar entre compute, memória e comunicação, dependendo da configuração.

- Foi desenvolvido um acelerador parametrizável para multiplicação de matrizes inteiras em FPGA.
- A arquitetura permite avaliar diferentes configurações de compute e memória.
- O fluxo experimental automatiza simulação, síntese, coleta de métricas e comparação.
- A separação modular facilita a evolução do projeto.

- Trocar a UART por uma interface de maior throughput.
- Criar uma interface padronizada para kernels.
- Permitir trocar o kernel sem reescrever controle e memória.
- Investigar se o gargalo principal está na memória, no protocolo ou no compute.
- Evoluir o uso de *double buffering*.

- Suportar múltiplos computadores ou clientes acessando o acelerador.
- Adicionar escalonamento de requisições.
- Adicionar isolamento entre usuários.
- Evoluir de um acelerador dedicado para uma infraestrutura compartilhável.

Referências I

Obrigado pela presença!
Perguntas?

Fernando Schettini

fernandoschettini@outlook.com